

TP GraphQL et MongoDB

Construire l'API d'un site marchand

Présentation

Au cours de ce TP, vous allez construire progressivement l'API GraphQL d'une boutique en ligne gérant des produits, des clients et des commandes, avec une base de données MongoDB.

Chaque exercice s'appuie sur le précédent : vous ferez évoluer le **même projet** du début à la fin. Ne repartez jamais de zéro.

Technologies utilisées :

- Node.js (version 20 ou supérieure, exigée par Apollo Server 5)
- Apollo Server 5 pour le serveur GraphQL
- MongoDB (en local ou sur MongoDB Atlas) et Mongoose

Conseil : faites un commit Git à la fin de chaque exercice (par exemple « exo1 catalogue produits »).

Exercice 1 : Le catalogue produits

Notions abordées : schéma SDL, types scalaires, type Query, resolvers, sélection de champs.

1. Créez le modèle Mongoose `src/models/Product.js` avec les champs suivants : `name` (texte, obligatoire), `description` (texte), `price` (nombre, obligatoire), `stock` (nombre, 0 par défaut), `category` (texte) et `createdAt` (date, valeur par défaut : date courante).
2. Insérez entre 8 et 10 produits de test, via Compass, mongosh ou un script `src/seed.js`. Variez les catégories (informatique, maison, sport...).
3. Dans `schema.js`, déclarez le type GraphQL suivant :

```
type Product {  
  id: ID!  
  name: String!  
  description: String  
  price: Float!  
  stock: Int!  
  category: String  
}
```

4. Déclarez le type Query avec deux points d'entrée :

```
type Query {  
  products: [Product!]!  
  product(id: ID!): Product  
}
```

5. Écrivez les resolvers correspondants dans `resolvers.js` en utilisant `Product.find()` et `Product.findById()`.
6. Testez dans Apollo Sandbox : récupérez tous les produits en demandant seulement leur nom et leur prix, puis un produit précis avec tous ses champs, et enfin un identifiant inexistant. Observez la réponse de l'API dans ce dernier cas.

Exercice 2 : Gérer le catalogue (mutations)

Notions abordées : type Mutation, types input, champs obligatoires, gestion des erreurs.

1. Déclarez un type input pour les produits :

```
input ProductInput {  
  name: String!  
  description: String  
  price: Float!  
  stock: Int  
  category: String  
}
```

2. Déclarez le type Mutation :

```
type Mutation {  
  createProduct(input: ProductInput!): Product!  
  updateProduct(id: ID!, input: ProductInput!): Product  
  deleteProduct(id: ID!): Boolean!  
}
```

3. Implémentez les trois resolvers. Pour la mise à jour, utilisez `findByIdAndUpdate` avec l'option `{ new: true }`.
4. Ajoutez des règles de validation : le prix doit être strictement positif, le stock ne peut pas être négatif, et une erreur explicite doit être renvoyée si l'identifiant n'existe pas. Utilisez `GraphQLError`, importé depuis le paquet `graphql`.
5. Testez successivement : la création d'un produit (vérifiez qu'il apparaît dans la liste), la modification de son prix, la création d'un produit au prix négatif, puis sa suppression.

Exercice 3 : Clients et commandes

Notions abordées : types imbriqués, resolvers de champs, références entre collections, logique métier.

1. Créez le modèle `Customer` avec les champs `firstName`, `lastName`, `email` (unique) et `address`.
2. Créez le modèle `Order` contenant : une référence vers le client (`ObjectId`), un tableau `items` dont chaque élément contient une référence vers un produit et une quantité, un statut ("PENDING" par défaut) et une date de création.
3. Déclarez les types GraphQL suivants :

```
type Customer {  
  id: ID!  
  firstName: String!  
  lastName: String!  
  email: String!  
  orders: [Order!]!  
}  
  
type OrderItem {  
  product: Product!  
  quantity: Int!  
}  
  
type Order {  
  id: ID!  
  customer: Customer!  
  items: [OrderItem!]!  
  total: Float!  
  status: String!
```

```
    createdAt: String!
  }
}
```

- Écrivez les resolvers de champs : `Order.customer` retrouve le client, `OrderItem.product` retrouve le produit, `Order.total` calcule la somme des prix multipliés par les quantités, et `Customer.orders` retrouve les commandes du client.
- Ajoutez les mutations `createCustomer(input: CustomerInput!)` et `createOrder(customerId: ID!, items: [OrderItemInput!]!)`.
- Dans `createOrder`, vérifiez que le client existe, que chaque produit existe et que son stock est suffisant. Si un seul stock est insuffisant, la commande entière doit être refusée. Sinon, décrémente les stocks puis enregistrez la commande.
- Testez une requête imbriquée comme celle ci :

```
query {
  customer(id: "...") {
    firstName
    orders {
      total
      items {
        quantity
        product { name price }
      }
    }
  }
}
```

Exercice 4 : Recherche, filtres, tri et pagination

Notions abordées : arguments, enums, input de filtre, traduction des arguments en requêtes MongoDB.

- Déclarez les enums et l'input de filtre suivants :

```
enum SortOrder { ASC DESC }
enum ProductSortField { PRICE NAME CREATED_AT }

input ProductFilter {
  category: String
  minPrice: Float
  maxPrice: Float
  search: String
  inStock: Boolean
}
```

- Modifiez la query `products` pour qu'elle accepte ces paramètres et renvoie une page de résultats :

```
products(
  filter: ProductFilter
  sortBy: ProductSortField
  order: SortOrder
  limit: Int = 10
  offset: Int = 0
): ProductPage!

type ProductPage {
  items: [Product!]!
  totalCount: Int!
  hasNextPage: Boolean!
}
```

- Dans le resolver, construisez dynamiquement l'objet de requête MongoDB : filtre sur la catégorie, utilisation de `$gte` et `$lte` pour la fourchette de prix, expression régulière insensible à la casse (ou index

texte avec \$text) pour la recherche, et condition stock supérieur à 0 pour inStock. Appliquez ensuite sort, skip et limit.

4. Testez les cas suivants : les produits de la catégorie informatique entre 50 et 300 euros triés par prix croissant, une recherche sur le mot « clavier », puis la deuxième page de résultats avec une limite de 3.

Exercice 5 : Fonctionnalités avancées

Réalisez obligatoirement la partie A, puis la partie B et/ou la partie C selon votre avancement.

Partie A : Statistiques de ventes

1. Déclarez les types et queries suivants :

```
type CategoryStat {
  category: String!
  revenue: Float!
  unitsSold: Int!
}

type ProductStat {
  product: Product!
  unitsSold: Int!
}

salesStats: [CategoryStat!]!
topProducts(limit: Int = 5): [ProductStat!]!
```

2. Implémentez les à l'aide du pipeline d'agrégation (Order.aggregate) en utilisant les étapes \$unwind, \$lookup, \$group, \$sort et \$limit.

3. Vérifiez vos résultats manuellement sur quelques commandes.

Partie B : Authentification et rôles

1. Ajoutez au modèle Customer un champ password (hashé avec bcrypt) et un champ role pouvant valoir CUSTOMER ou ADMIN.

2. Créez une mutation login(email, password) renvoyant un jeton JWT, généré avec la bibliothèque jsonwebtoken.

3. Dans la fonction context d'Apollo Server, lisez l'en tête Authorization, vérifiez le jeton et placez l'utilisateur connecté dans le contexte.

4. Protégez les mutations createProduct, updateProduct et deleteProduct afin qu'elles soient réservées aux administrateurs. Renvoyez une erreur UNAUTHENTICATED ou FORBIDDEN selon le cas.

5. Ajoutez une query me renvoyant l'utilisateur connecté et ses commandes.

Partie C : Temps réel avec les subscriptions

1. Mettez en place les subscriptions à l'aide de la bibliothèque graphql-ws et d'un serveur combinant HTTP et WebSocket.

2. Déclarez le type Subscription contenant le champ orderCreated: Order!

3. Publiez un événement à chaque appel de la mutation createOrder.

4. Ouvrez deux onglets : abonnez vous dans le premier, créez une commande dans le second, puis observez la notification reçue.

Livrable attendu

- Le dépôt Git du projet, avec au minimum un commit par exercice.
- Un fichier README.md contenant les instructions de lancement, un exemple de requête testée pour chaque exercice et vos réponses aux questions de réflexion.
- Le script de seed permettant de remplir la base de données.

Ressources utiles

- Documentation GraphQL : <https://graphql.org/learn/>
- Documentation Apollo Server : <https://www.apollographql.com/docs/apollo-server/>
- Documentation Mongoose : <https://mongoosejs.com/docs/>
- Agrégation MongoDB : <https://www.mongodb.com/docs/manual/aggregation/>